

Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology
Department of Computer Science & Engineering



Project Report

Course Code: CSE 3206

Course Title: Software Engineering Sessional

Lab 3 – Design Pattern Analysis, Implementation and Code Review

Authors

Abdullah Al Kafi
2203139

Sadaf Rahman
2203140

Sakila Akter
2203141

Group #07, Section C (1st 30)
Design Patterns: Iterator, Mediator

Supervisor

Farjana Parvin
Assistant Professor
Dept. of CSE, RUET

Date: 12/10/2026

Contents

Introduction	2
1 Iterator Pattern	3
1.1 Pattern Name	3
1.2 Category	3
1.3 Intent.....	3
1.4 Problem Statement	3
1.5 Motivation	4
1.6 Pattern Structure	5
1.7 Class Responsibilities	6
1.8 Code Implementation	6
1.9 Execution Flow	8
1.10 Advantages	9
1.11 Limitations	9
1.12 Real-life Applications.....	10
1.13 Industry Examples.....	10
1.14 Demonstration	10
1.15 Conclusion.....	10
2 Mediator Pattern	12
2.1 Pattern Name	12
2.2 Category	12
2.3 Intent.....	12
2.4 Problem Statement	12
2.5 Motivation	13
2.6 Pattern Structure	14
2.7 Class Responsibilities	15
2.8 Code Implementation	15
2.9 Execution Flow	17
2.10 Advantages	18
2.11 Limitations	18
2.12 Real-life Applications.....	18
2.13 Industry Examples.....	19
2.14 Demonstration	19
2.15 Conclusion.....	19
3 Code Review	20
3.1 Review Checklist.....	20
3.2 Suggested Improvements	20
4 Version Control (GitHub)	21
4.1 Repository	21
4.2 Repository Structure.....	21
4.3 Workflow	22
4.4 Command-Line Usage.....	23
4.5 Repository Screenshot.....	23
5 Comparison: Iterator vs Mediator	24
6 Conclusion.....	24
References	25

Introduction

This report is prepared for **Lab 3: Design Pattern Analysis, Implementation and Code Review** of the course **CSE 3206 (Software Engineering Sessional)**. Our group (**Group 07**) was given two design patterns: **Iterator** and **Mediator**. Both are *behavioral* patterns. Behavioral patterns are about how objects talk to each other and how they share work.

For each pattern we follow the 15-point documentation format given in the lab sheet (pattern name, category, intent, problem, motivation, UML structure, class responsibilities, code, execution flow, advantages, limitations, real-life applications, industry examples, demonstration and conclusion). We picked one small, everyday example for each pattern so that the idea is easy to see:

Pattern	Our example	Source files
Iterator	RUET cafeteria menu. A waiter prints a Breakfast Menu (stored in an array) and a Lunch Menu (stored in a vector) with the same loop.	<code>without_iterator.cpp</code> <code>iterator.cpp</code>
Mediator	Group chat room. Kafi, Sadaf and Sakila send messages through a chat room instead of talking to each other directly.	<code>without_mediator.cpp</code> <code>mediator.cpp</code>

Table 1: The two patterns and the examples used in this report.

For every pattern we show **two programs**: first a short program *without* the pattern (to show the problem), and then the program *with* the pattern (the solution). All programs are written in **C++** (C++11 or newer) and compiled with **g++**. Every output shown in this report was produced by actually running the programs.

Team members

Name	Student ID	GitHub username
Abdullah Al Kafi	2203139	<code>Kafi2611</code>
Sadaf Rahman	2203140	<code>sadaf532</code>
Sakila Akter	2203141	<code>SakilaAkter</code>

Table 2: Group 07 members.

A quick idea of both patterns

Iterator: “Give me the next item, I don’t care how you store them.” The collection hands out a small helper (the iterator) that walks through the items one by one.

Mediator: “Don’t talk to each other, talk to me.” One middle object (the mediator) carries all the communication between a group of objects.

1 Iterator Pattern

1.1 Pattern Name

Iterator Pattern (also known as **Cursor**).

1.2 Category

Behavioral Design Pattern. It is behavioral because it decides how two objects – a client and a collection – work together when the client visits the items one after another.

1.3 Intent

“Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.” — Gang of Four (GoF)

In simple words: a collection (array, vector, list, tree ...) gives us a helper object called an **iterator**. With only two functions – `hasNext()` and `next()` – we can visit every item one by one, and we never need to know how the items are stored inside.

1.4 Problem Statement

In the RUET cafeteria, a waiter has to read out two menus to the students:

- The **Breakfast Menu** keeps its items in a fixed **array**.
- The **Lunch Menu** keeps its items in a **vector**.

The waiter only wants to say every item, one by one. But without a common way to walk through the items, the waiter’s code must know how each menu stores its data. Code 1 shows this normal (bad) way.

Code 1: Without Iterator Pattern — without_iterator.cpp

```

1  // WITHOUT Iterator Pattern (the problem)
2  #include <iostream>
3  #include <string>
4  #include <vector>
5  using namespace std;
6
7  class BreakfastMenu {
8  public:
9      string items[3] = {"Paratha", "Egg Bhaji", "Tea"}; // must be public
10 };
11
12 class LunchMenu {
13 public:
14     vector<string> items = {"Rice", "Chicken Curry", "Dal"}; // must be public
15 };
16
17 // The waiter must know HOW each menu stores its items
18 void printMenus(BreakfastMenu& b, LunchMenu& l) {
19     for (int i = 0; i < 3; i++) // Loop for the array
20         cout << " - " << b.items[i] << endl;
21     for (int i = 0; i < (int)l.items.size(); i++) // another loop for the vector
22         cout << " - " << l.items[i] << endl;
23     // a new menu type (e.g. Linked List) = one more loop here
24 }
25
26 int main() {
27     BreakfastMenu b;
28     LunchMenu l;
29     printMenus(b, l);
30     return 0;
31 }

```

Problems in Code 1:

1. The menu items must be **public**, so any code can read or change them (no encapsulation).
2. The waiter needs a **different loop for each menu** – one for the array and one for the vector.
3. If the cafeteria adds a **Dinner Menu** stored in a linked list, we must open `printMenus()` and add another loop. Old, working code keeps changing.

1.5 Motivation

Why is the normal implementation poor? In Code 1 the waiter does two jobs: it prints the items *and* it knows the storage details of every menu. This breaks three good design rules:

- **Encapsulation** – private data has to be made public.
- **Single Responsibility Principle (SRP)** – the waiter has more than one reason to change.
- **Open/Closed Principle (OCP)** – every new menu forces us to edit existing code.

The Iterator Pattern solves this. Each menu gives out an **iterator**. The iterator has only two functions: `hasNext()` (“is there another item?”) and `next()` (“give me the next item”). The waiter talks only to the iterator, so **one loop works for every menu** (Figure 1).

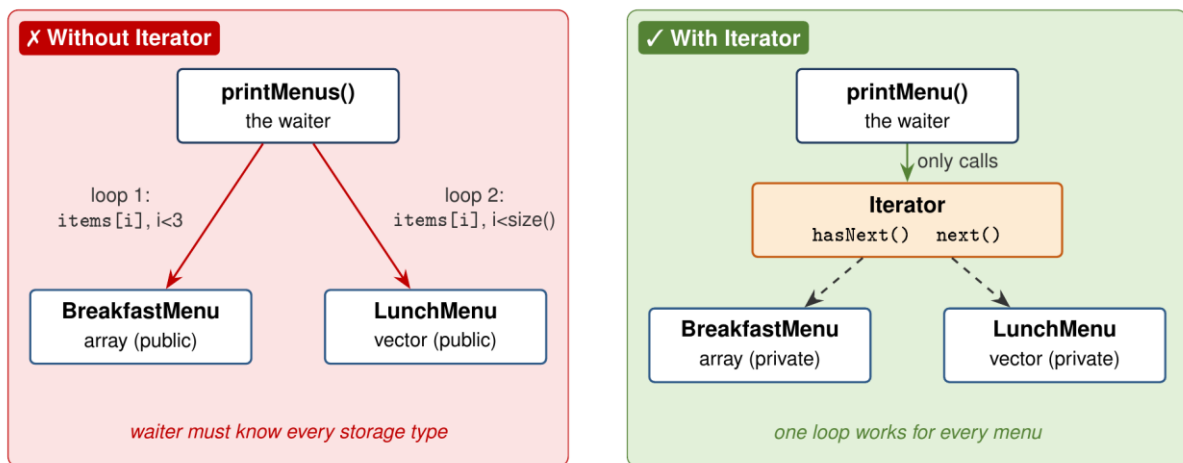


Figure 1: Without the pattern the waiter must know every storage type (left). With the pattern the waiter only calls `hasNext()` and `next()` (right).

1.6 Pattern Structure

Figure 2 shows the UML class diagram of our example. **Orange** boxes are interfaces (abstract classes in C++), **blue** boxes are concrete classes and the **green** box is the client. A dashed arrow with an empty triangle means “implements”.

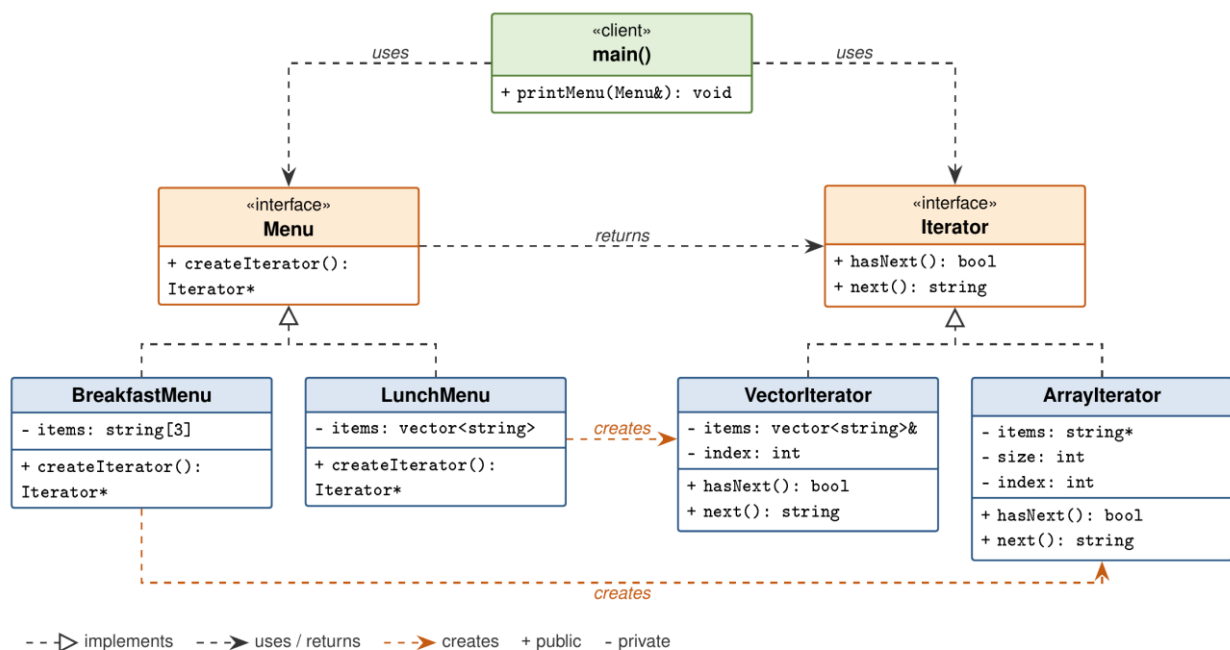


Figure 2: UML class diagram of the Iterator Pattern (cafeteria menu example).

1.7 Class Responsibilities

Class	Role in the pattern	Responsibility
Iterator	Iterator (interface)	Declares <code>hasNext()</code> and <code>next()</code> . The client uses only these two functions.
ArrayIterator	Concrete Iterator	Walks through an array using an index. Remembers the current position.
VectorIterator	Concrete Iterator	Walks through a vector using an index. Remembers the current position.
Menu	Aggregate (interface)	Declares <code>createIterator()</code> . Every menu must be able to give an iterator.
BreakfastMenu	Concrete Aggregate	Stores its items in a private array and returns an <code>ArrayIterator</code> .
LunchMenu	Concrete Aggregate	Stores its items in a private vector and returns a <code>VectorIterator</code> .
printMenu() / main()	Client	Prints any menu by using only <code>hasNext()</code> and <code>next()</code> .

Table 3: Classes of the Iterator example and their jobs.

1.8 Code Implementation

Code 2 is the complete program with the Iterator Pattern. To make it easy to follow, we show it in **four small parts** and explain each part right after it. The line numbers continue from one part to the next, so together the four parts are the full `iterator.cpp` file.

Code 2 – Part A: the two interfaces (lines 1–20)

```

1  // Iterator Pattern - RUET Cafeteria Menu
2  #include <iostream>
3  #include <string>
4  #include <vector>
5  using namespace std;
6
7  // Iterator interface
8  class Iterator {
9  public:
10     virtual bool hasNext() = 0;
11     virtual string next() = 0;
12     virtual ~Iterator() {}
13 };
14
15 // Aggregate interface
16 class Menu {
17 public:
18     virtual Iterator* createIterator() = 0;
19     virtual ~Menu() {}
20 };

```

- **Lines 8–13 – Iterator interface.** Two pure virtual functions: `hasNext()` and `next()`. Every iterator must have them. The virtual destructor lets us safely delete any iterator through an `Iterator*` pointer.
- **Lines 16–20 – Menu interface (the aggregate).** Every menu must have `createIterator()`, which gives back an iterator for that menu.

Code 2 – Part B: the two concrete iterators (lines 22–41)

```

22 // Concrete Iterator 1: walks through an array
23 class ArrayIterator : public Iterator {
24     string* items;
25     int size;
26     int index = 0;
27 public:
28     ArrayIterator(string* items, int size) : items(items), size(size) {}
29     bool hasNext() override { return index < size; }
30     string next() override { return items[index++]; }
31 };
32
33 // Concrete Iterator 2: walks through a vector
34 class VectorIterator : public Iterator {
35     vector<string>& items;
36     int index = 0;
37 public:
38     VectorIterator(vector<string>& items) : items(items) {}
39     bool hasNext() override { return index < (int)items.size(); }
40     string next() override { return items[index++]; }
41 };

```

- **Lines 23–31 – ArrayIterator.** Keeps a pointer to the array, its size, and the current index (starts at 0). `hasNext()` checks `index < size`. `next()` returns the current item and then moves index forward (`index++`).
- **Lines 34–41 – VectorIterator.** Same idea, but for a vector, so it uses `items.size()` instead of a stored size.

Code 2 – Part C: the two menus (lines 43–59)

```

43 // Concrete Aggregate 1: keeps items in an array
44 class BreakfastMenu : public Menu {
45     string items[3] = {"Paratha", "Egg Bhaji", "Tea"};
46 public:
47     Iterator* createIterator() override {
48         return new ArrayIterator(items, 3);
49     }
50 };
51
52 // Concrete Aggregate 2: keeps items in a vector
53 class LunchMenu : public Menu {
54     vector<string> items = {"Rice", "Chicken Curry", "Dal"};
55 public:
56     Iterator* createIterator() override {
57         return new VectorIterator(items);
58     }
59 };

```

- **Lines 44–50 – BreakfastMenu.** Its items are in a **private array**. `createIterator()` returns a new `ArrayIterator` for that array.
- **Lines 53–59 – LunchMenu.** Its items are in a **private vector**. `createIterator()` returns a new `VectorIterator`.
- Each menu knows its own storage, so each menu creates the right iterator. Nobody outside the menu sees the array or the vector.

Code 2 – Part D: the client (lines 61–81)

```

61 // Client: works with ANY menu, does not know how items are stored
62 void printMenu(Menu& menu) {
63     Iterator* it = menu.createIterator();
64     while (it->hasNext()) {
65         cout << "  - " << it->next() << endl;
66     }
67     delete it;
68 }
69
70 int main() {
71     BreakfastMenu breakfast;
72     LunchMenu lunch;
73
74     cout << "Breakfast Menu:" << endl;
75     printMenu(breakfast);
76
77     cout << "Lunch Menu:" << endl;
78     printMenu(lunch);
79
80     return 0;
81 }

```

- **Lines 62–68 – printMenu().** Asks the menu for an iterator, loops while hasNext() is true, prints next(), and finally deletes the iterator. It takes a Menu&, so it works for **any** menu.
- **Lines 70–81 – main().** Creates both menus and calls the **same** printMenu() for both of them.

Key point

printMenu() never uses [i], size(), “array” or “vector”. It only knows Menu and Iterator. That is the whole idea of the pattern.

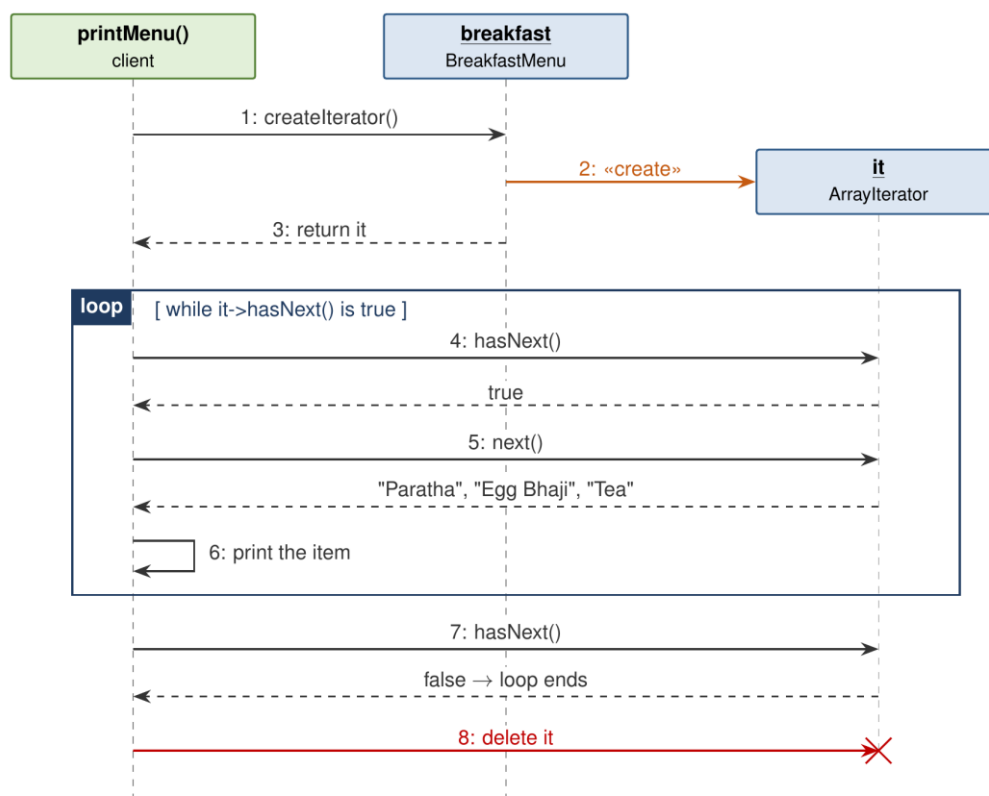
1.9 Execution Flow

What happens when main() calls printMenu(breakfast):

1. main() creates a BreakfastMenu object and calls printMenu(breakfast).
2. printMenu() calls breakfast.createIterator().
3. BreakfastMenu creates a new ArrayIterator (with index = 0) and returns it.
4. The loop calls hasNext(): 0 < 3 is true, so next() returns “Paratha” and index becomes 1.
5. The same happens for “Egg Bhaji” and “Tea”.
6. Now index = 3, so hasNext() returns false and the loop stops. printMenu() deletes the iterator.
7. main() then calls printMenu(lunch). This time LunchMenu returns a VectorIterator, but printMenu() works in exactly the same way.

Round	index (before)	hasNext()	next() returns	index (after)
1	0	true	Paratha	1
2	1	true	Egg Bhaji	2
3	2	true	Tea	3
4	3	false	— (loop ends)	3

Table 4: State of the ArrayIterator while printing the Breakfast Menu.

Figure 3: Sequence diagram of `printMenu(breakfast)`.

1.10 Advantages

- **Hides the inside of a collection.** Items stay private; the client never sees the array or the vector.
- **One loop for all collections.** The same `printMenu()` works for any menu.
- **Easy to add new collections (OCP).** A new `DinnerMenu` stored in a linked list only needs its own iterator class; `printMenu()` does not change.
- **Each class has one job (SRP).** The menu stores items, the iterator walks, the client prints.
- **Many walks at the same time.** Each iterator keeps its own `index`, so two iterators over the same menu do not disturb each other.
- **Different ways to walk.** We can add a reverse iterator or a “vegetarian only” iterator without touching the menu data.

1.11 Limitations

- **Too much for tiny programs.** For one simple array, a normal `for` loop is shorter.

- **More classes.** Every new storage type needs its own iterator class.
- **Changing the collection during a loop** (adding or deleting items) can break the iterator.
- **Usually forward only.** Going back or jumping to a position needs extra functions.
- **Small speed cost.** An extra object and virtual function calls compared with direct indexing.
- **Memory care.** In our code the iterator is created with `new`, so the client must remember to `delete` it (a smart pointer can do this automatically).

1.12 Real-life Applications

- **TV remote:** the channel up/down button moves to the next channel; we don't need to know how channels are stored.
- **Music player “Next” button:** the same button works for songs on the phone or songs from the internet.
- **Token system** at a bank or hospital: “next token, please” – people are served one by one.
- **Social media feed:** scrolling shows the next posts, which are loaded little by little.
- **A bookmark in a book:** it remembers where we are, just like an iterator remembers its index.

1.13 Industry Examples

- **C++ STL:** every container (`vector`, `list`, `map`, `set`) gives `begin()` and `end()` iterators, and the range-based `for` loop uses them.
- **Java:** the `Iterator` and `Iterable` interfaces power the `for-each` loop.
- **Python:** `__iter__()` / `__next__()` and generators with `yield`.
- **Databases:** cursors (for example `JDBC ResultSet.next()`) read query results row by row.
- **File systems:** C++17 `std::filesystem::directory_iterator` walks through the files of a folder.
- **JavaScript:** `Symbol.iterator` and the `for...of` loop.

1.14 Demonstration

Compiling and running Code 2 gives the following output:

Output of Code 2 (iterator.cpp)

```
Terminal
$ g++ iterator.cpp -o iterator && ./iterator
Breakfast Menu:
- Paratha
- Egg Bhaji
- Tea
Lunch Menu:
- Rice
- Chicken Curry
- Dal
```

Both menus were printed by the **same** `printMenu()` function, even though one menu uses an array and the other uses a vector. The client did not need to know the difference.

1.15 Conclusion

The Iterator Pattern gives every collection the same simple “next item” interface. In our cafeteria example, one `printMenu()` function printed an array-based menu and a vector-based menu without knowing how they store their items. The menu data stayed private, and a new menu can be added without touching the client code. This idea is so useful that C++, Java, Python and JavaScript all have iterators built into the language.

2 Mediator Pattern

2.1 Pattern Name

Mediator Pattern (also called **Intermediary** or **Controller**).

2.2 Category

Behavioral Design Pattern. It is behavioral because it controls how a group of objects communicate with each other, and who decides what happens next.

2.3 Intent

“Define an object that encapsulates how a set of objects interact. Mediator promotes loose coupling by keeping objects from referring to each other explicitly, and it lets you vary their interaction independently.” — Gang of Four (GoF)

In simple words: put **one object in the middle**. All the other objects talk only to this middle object (the **mediator**), never directly to each other.

2.4 Problem Statement

Our group has a chat with three members: **Kafi**, **Sadaf** and **Sakila**. When one person sends a message, everyone else should get it. If users talk to each other directly, every user must keep a list of all the other users. Code 3 shows this normal (bad) way.

Code 3: Without Mediator Pattern — without_mediator.cpp

```

1  // WITHOUT Mediator Pattern (the problem)
2  #include <iostream>
3  #include <string>
4  #include <vector>
5  using namespace std;
6
7  class User {
8      string name;
9      vector<User*> friends;    // every user must keep every other user
10 public:
11     User(string name) : name(name) {}
12
13     void addFriend(User* u) { friends.push_back(u); }
14
15     void send(const string& msg) {
16         cout << name << " sends: " << msg << endl;
17         for (User* f : friends)
18             f->receive(msg, name);
19     }
20
21     void receive(const string& msg, const string& from) {
22         cout << " " << name << " got from " << from << ": " << msg << endl;
23     }
24 };
25
26 int main() {
27     User kafi("Kafi"), sadaf("Sadaf"), sakila("Sakila");
28
29     // 3 users already need 6 links; 10 users would need 90!
30     kafi.addFriend(&sadaf);    kafi.addFriend(&sakila);
31     sadaf.addFriend(&kafi);    sadaf.addFriend(&sakila);
32     sakila.addFriend(&kafi);    sakila.addFriend(&sadaf);
33
34     kafi.send("Lab report is ready!");
35     return 0;
36 }

```

Problems in Code 3:

1. **Too many links.** Every pair of users must be connected, in both directions. 3 users already need 6 addFriend() calls; 10 users would need 90.
2. **Hard to add a member.** A new member must be added to the list of every old member.
3. **Rules are spread everywhere.** The “who gets the message” logic is inside every User. To add a rule (for example, block a user or save chat history) we must change the User class.
4. **Users cannot be reused alone** because they are tightly tied to each other.

2.5 Motivation

Why is the normal implementation poor? It creates a many-to-many web of objects that is hard to understand, test and change. In a real WhatsApp or Messenger group, our phone does not connect to every member’s phone. It sends the message to the **server**, and the server delivers it to everyone. The Mediator Pattern does the same in code:

- User objects (the **colleagues**) know only the ChatMediator.
- ChatRoom (the **mediator**) keeps the list of users and decides who receives each message.

The tangled web becomes a simple **star** (Figure 4): n users need only n links, and all the rules live in one class.

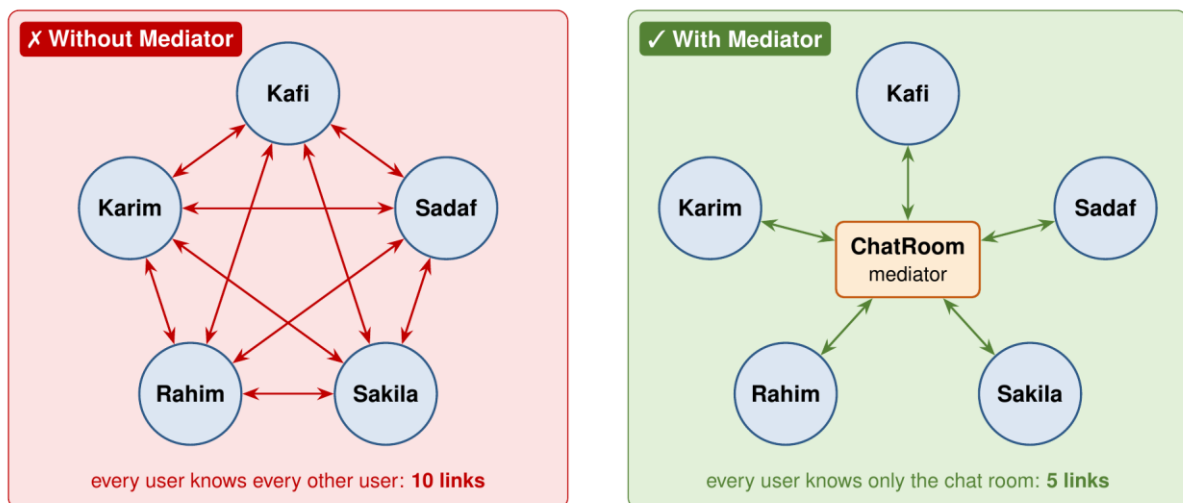


Figure 4: Five users without a mediator need 10 links (left). With a chat room in the middle, each user needs only one link (right).

2.6 Pattern Structure

Figure 5 shows the UML class diagram of our example. `User` depends only on the `ChatMediator` interface. `ChatRoom` implements that interface and keeps the list of users. Notice that there is **no link from User to User**.

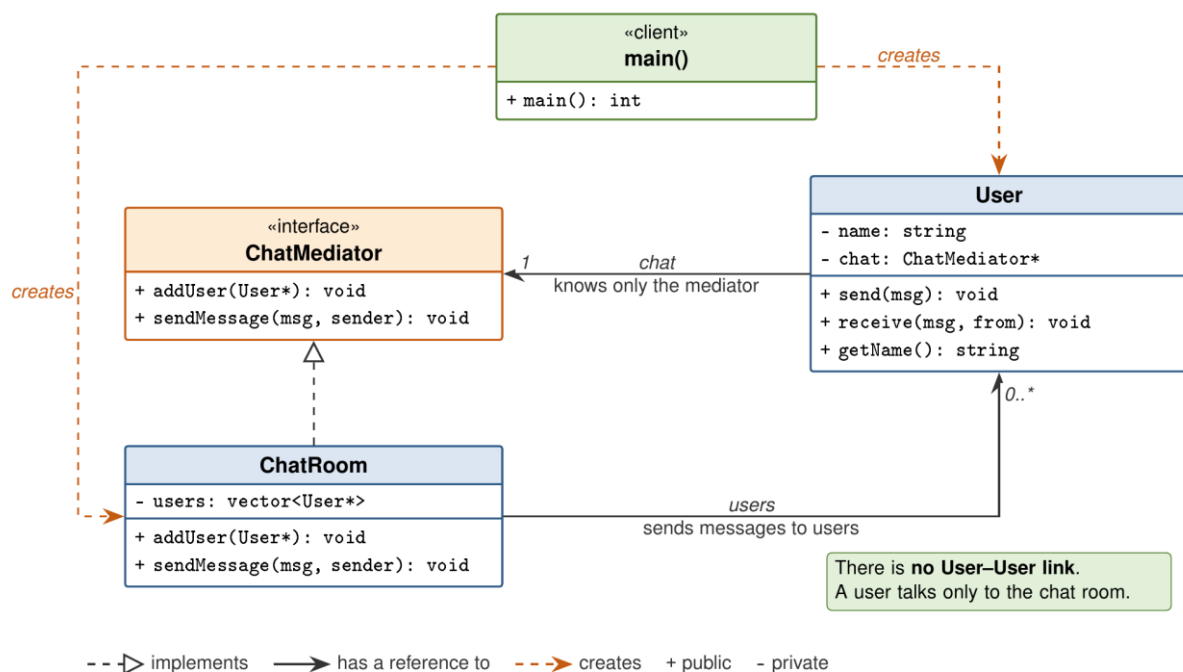


Figure 5: UML class diagram of the Mediator Pattern (group chat example).

2.7 Class Responsibilities

Class	Role in the pattern	Responsibility
ChatMediator	Mediator (interface)	Declares <code>addUser()</code> and <code>sendMessage()</code> . Users depend on this interface, not on <code>ChatRoom</code> .
ChatRoom	Concrete Mediator	Keeps the list of users. When a message arrives, it sends it to every user except the sender .
User	Colleague	Has a name and a pointer to the mediator. <code>send()</code> gives the message to the mediator; <code>receive()</code> prints a message from someone else. It never holds a pointer to another <code>User</code> .
main()	Client	Creates the chat room and the users, adds the users to the room and starts the conversation.

Table 5: Classes of the Mediator example and their jobs.

2.8 Code Implementation

Code 4 is the complete program with the Mediator Pattern. Like Code 2, we show it in **four small parts** and explain each part right after it. The line numbers continue from one part to the next, so together the four parts are the full `mediator.cpp` file.

Code 4 – Part A: the mediator interface (lines 1–15)

```

1  // Mediator Pattern - Group Chat Room
2  #include <iostream>
3  #include <string>
4  #include <vector>
5  using namespace std;
6
7  class User; // forward declaration
8
9  // Mediator interface
10 class ChatMediator {
11 public:
12     virtual void addUser(User* user) = 0;
13     virtual void sendMessage(const string& msg, User* sender) = 0;
14     virtual ~ChatMediator() {}
15 };

```

- **Line 7 – forward declaration.** `class User;` tells the compiler that a class named `User` exists, so `ChatMediator` can use `User*` before `User` is fully written.
- **Lines 10–15 – `ChatMediator` interface.** Two pure virtual functions: `addUser()` (join the chat) and `sendMessage()` (pass a message to the others).

Code 4 – Part B: the User class, our colleague (lines 17–34)

```

17 // Colleague: a user knows ONLY the mediator, not other users
18 class User {
19     string name;
20     ChatMediator* chat;
21 public:
22     User(string name, ChatMediator* chat) : name(name), chat(chat) {}
23
24     string getName() { return name; }
25
26     void send(const string& msg) {
27         cout << name << " sends: " << msg << endl;
28         chat->sendMessage(msg, this);
29     }
30
31     void receive(const string& msg, const string& from) {
32         cout << " " << name << " got from " << from << ": " << msg << endl;
33     }
34 };

```

- **Lines 19–20 – data.** A user stores only its `name` and a `chat` pointer to the mediator. There is **no** list of other users.
- **Lines 26–29 – `send()`.** Prints the message and hands it to the mediator with `chat->sendMessage(msg, this)`. Here `this` tells the room who the sender is.
- **Lines 31–33 – `receive()`.** Called by the room; it prints a message that came from someone else.

Code 4 – Part C: the ChatRoom, our concrete mediator (lines 36–51)

```

36 // Concrete Mediator: decides who gets the message
37 class ChatRoom : public ChatMediator {
38     vector<User*> users;
39 public:
40     void addUser(User* user) override {
41         users.push_back(user);
42     }
43
44     void sendMessage(const string& msg, User* sender) override {
45         for (User* u : users) {
46             if (u != sender) { // do not send back to the sender
47                 u->receive(msg, sender->getName());
48             }
49         }
50     }
51 };

```

- **Line 38 – `users`.** The room keeps a `vector<User*>` of all members.
- **Lines 40–42 – `addUser()`.** Adds a user to the list.
- **Lines 44–50 – `sendMessage()`.** Loops over all users and calls `receive()` on everyone **except the sender** (the check on line 46). This rule lives only here, in the mediator.

Code 4 – Part D: main(), our client (lines 53–68)

```

53 int main() {
54     ChatRoom room;
55
56     User kafi("Kafi", &room);
57     User sadaf("Sadaf", &room);
58     User sakila("Sakila", &room);
59
60     room.addUser(&kafi);
61     room.addUser(&sadaf);
62     room.addUser(&sakila);
63
64     kafi.send("Lab report is ready!");
65     sadaf.send("Great, I will check the code.");
66
67     return 0;
68 }

```

- **Lines 54–58.** Create one `ChatRoom` and three users. Each user gets the address of the **same** room (`&room`).
- **Lines 60–62.** Add all three users to the room.
- **Lines 64–65.** Kafi and then Sadaf send a message. They call only their own `send()`; the room does the rest.

Key point

Search the `User` class: there is no `User*` member and no call to another user. Every message goes **through** `ChatRoom`. That is the whole idea of the pattern.

2.9 Execution Flow

What happens when Kafi sends a message:

1. `main()` creates one `ChatRoom` and three `User` objects. Each user gets a pointer to the **same** room.
2. `main()` adds all three users to the room, so `users = [Kafi, Sadaf, Sakila]`.
3. `kafi.send("Lab report is ready!")` prints “Kafi sends: ...” and calls `room.sendMessage(msg, &kafi)`.
4. The room loops over its users. Kafi is the sender, so he is **skipped**.
5. The room calls `sadaf.receive(msg, "Kafi")` and `sakila.receive(msg, "Kafi")`.
6. Next, `sadaf.send(...)` works the same way; this time Kafi and Sakila receive the message.

Message	Sender	Room checks each user	Received by
“Lab report is ready!”	Kafi	Kafi (skip), Sadaf ✓, Sakila ✓	Sadaf, Sakila
“Great, I will check the code.”	Sadaf	Kafi ✓, Sadaf (skip), Sakila ✓	Kafi, Sakila

Table 6: What the `ChatRoom` does for each message.

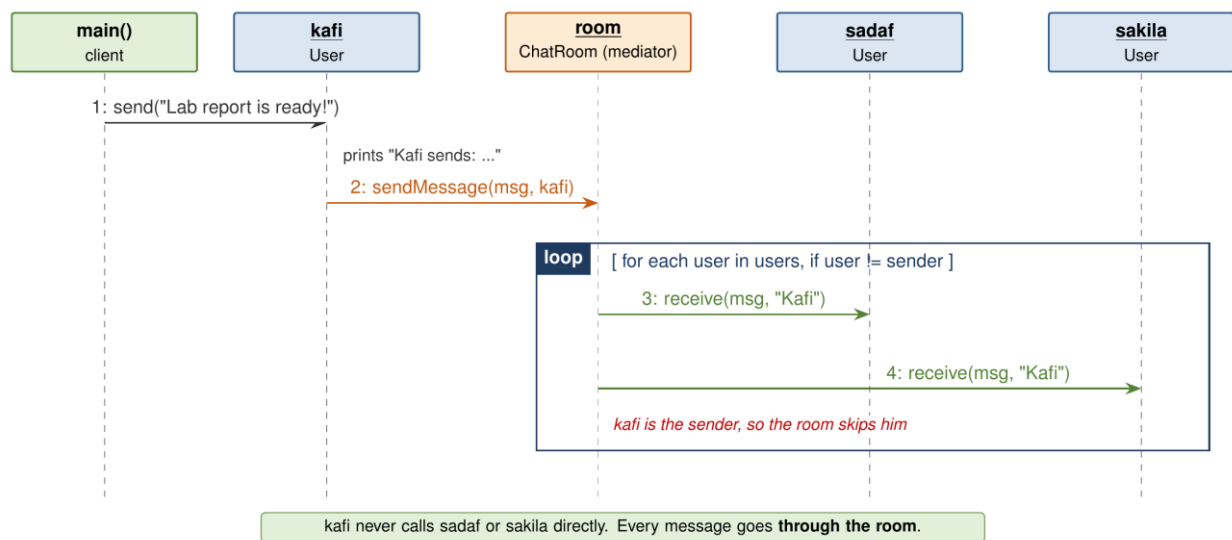


Figure 6: Sequence diagram of `kafi.send("Lab report is ready!")`.

2.10 Advantages

- **Loose coupling.** A User knows only the mediator, never the other users.
- **Fewer links.** A many-to-many web of $n(n-1)/2$ links becomes a star of only n links.
- **All rules in one place.** To add a rule (block a user, save chat history, filter bad words) we change only ChatRoom.
- **Easy to add members.** A new user simply joins the room; old users do not change.
- **Reusable colleagues.** User works with any class that implements ChatMediator (for example a private chat or a class-announcement room).
- **Follows SRP and OCP.** Communication logic is separated from the users, and new mediators can be added without editing User.

2.11 Limitations

- **The mediator can become a “god object”.** If we keep adding rules, ChatRoom becomes very big and hard to maintain.
- **Single point of failure.** If the mediator has a bug or stops, nobody can talk.
- **Possible bottleneck.** Every message must pass through one object.
- **Harder to trace.** A call goes User → ChatRoom → other Users, so debugging takes more steps.
- **Overkill for two objects.** If only two objects talk, a mediator only adds extra code.

2.12 Real-life Applications

- **Air traffic control tower:** pilots talk only to the tower, not to each other.
- **Class Representative (CR):** the teacher gives a notice to the CR, and the CR passes it to all students.
- **Traffic police at a road crossing:** cars follow the police signal instead of talking to each other.
- **Ride-sharing apps (Pathao, Uber):** the app matches riders and drivers; they do not need to find each other.

- **WhatsApp / Messenger groups:** the server delivers every message to all members.

2.13 Industry Examples

- **GUI dialog boxes:** a dialog object controls its buttons, text boxes and check boxes (for example, it enables the “OK” button only when all fields are filled). This is the classic example in the GoF book.
- **MVC frameworks:** the Controller sits between the View and the Model, so they do not depend on each other.
- **MediatR library (.NET):** requests are sent to one mediator, which passes them to the right handler.
- **Message brokers (RabbitMQ, Apache Kafka):** services publish messages to the broker instead of calling each other directly.
- **Chat servers (Slack, Discord, WhatsApp):** clients send messages to a server, which forwards them to the other members.

2.14 Demonstration

Compiling and running Code 4 gives the following output:

Output of Code 4 (mediator.cpp)

```
Terminal
$ g++ mediator.cpp -o mediator && ./mediator
Kafi sends: Lab report is ready!
  Sadaf got from Kafi: Lab report is ready!
  Sakila got from Kafi: Lab report is ready!
Sadaf sends: Great, I will check the code.
  Kafi got from Sadaf: Great, I will check the code.
  Sakila got from Sadaf: Great, I will check the code.
```

Each message reached every other member, and the sender did not get his or her own message back. No `User` object ever called another `User` – everything went through `ChatRoom`.

2.15 Conclusion

The Mediator Pattern replaces a messy web of object-to-object links with one central object. In our chat example, three users talked to each other without holding a single pointer to one another. All the message rules live in `ChatRoom`, so changing a rule (for example, blocking a user) means editing just one class. The price is that the mediator must be kept small and well organised, or it can grow too large.

3 Code Review

The lab sheet asks every group to review code with a 10-point checklist. We used the same checklist on our own two programs. **Good** means the point is handled well; **Can improve** means there is a small issue we found.

3.1 Review Checklist

Checklist item	Iterator code (iterator.cpp)	Mediator code (mediator.cpp)
Naming Convention	Good – classes use PascalCase (BreakfastMenu), functions use camelCase (hasNext).	Good – clear names: ChatRoom, sendMessage, receive.
SOLID Principles	Good – SRP (menu stores, iterator walks, client prints); OCP (new menu needs no change in printMenu); DIP (client depends on interfaces).	Good – SRP (user talks, room routes); OCP (new mediator type without changing User); DIP (User depends on ChatMediator).
Readability	Good – short classes, one comment above each part.	Good – short classes, comments explain each role.
Object Interaction	Good – client → menu → iterator; client only calls hasNext() / next().	Good – user → room → other users; no direct user-to-user call.
Pattern Correctness	Good – all GoF roles present: Iterator, Concrete Iterator, Aggregate, Concrete Aggregate, Client.	Good – all GoF roles present: Mediator, Concrete Mediator, Colleague, Client.
Reusability	Good – ArrayIterator / VectorIterator can be reused by any class that stores strings.	Good – User works with any ChatMediator.
Exception Handling	Can improve – next() does not check hasNext(); calling it after the end reads outside the array.	Can improve – the room does not check that the sender is a member, or that chat is not null.
Documentation	Good – comments name each pattern role; could add short comments on each function.	Good – comments name each pattern role; could add short comments on each function.
Maintainability	Good – adding a menu touches only new classes.	Good – message rules live in one class (ChatRoom).
Efficiency	Good – O(n) walk, no copying of items. Can improve – raw new/delete for each walk.	Good – O(n) per message. Can improve – raw pointers mean users must live longer than the room.

Table 7: Code review of our two programs using the lab checklist.

3.2 Suggested Improvements

We kept the main code short on purpose so that the pattern is easy to see. For a real project we would add these small changes:

1. **Safe next()** – throw an exception at the end: `if (!hasNext()) throw out_of_range("No more items");`

2. **Smart pointers** – return `unique_ptr<Iterator>` from `createIterator()`, so the iterator is deleted automatically.
3. **Member check** – in `ChatRoom::sendMessage()`, ignore messages from users who were never added to the room.
4. **const-correctness** – write `string getName() const` because the function does not change the object.

4 Version Control (GitHub)

4.1 Repository

We use **Git** for version control and **GitHub** to keep our code online. The repository is owned by Kafi, and the other two members are added as collaborators. Every member pushed his or her own part from a separate **branch** and opened a **pull request**; Kafi reviewed the pull requests and merged them into `main`.

<https://github.com/Kafi2611/CSE3206-Lab3-Iterator-Mediator>

Member	GitHub	Role	Branch	Files pushed
Abdullah Al Kafi	Kafi2611	Owner, reviewer	main kafi-diagrams-report	README.md, .gitignore, both “without pattern” codes, Diagrams/, Report/, Code_Review.md
Sadaf Rahman	sadaf532	Collaborator	sadaf-iterator	Iterator/iterator.cpp
Sakila Akter	SakilaAkter	Collaborator	sakila-mediator	Mediator/mediator.cpp

Table 8: Who did what in the GitHub repository.

4.2 Repository Structure

Folder structure of the repository

```
CSE3206-Lab3-Iterator-Mediator/
├── Iterator/
│   ├── without_iterator.cpp    # Code 1 - pushed by Kafi
│   └── iterator.cpp           # Code 2 - pushed by Sadaf
├── Mediator/
│   ├── without_mediator.cpp    # Code 3 - pushed by Kafi
│   └── mediator.cpp           # Code 4 - pushed by Sakila
├── Diagrams/                  # UML diagrams - pushed by Kafi
├── Report/                    # this report - pushed by Kafi
├── Code_Review.md             # code review - pushed by Kafi
├── .gitignore                 # files Git should ignore (.exe etc.)
└── README.md                  # how to compile and run - Kafi
```

4.3 Workflow

Figure 7 shows how a change travels from our computer to GitHub, and Figure 8 shows how our three branches were joined into main with pull requests.

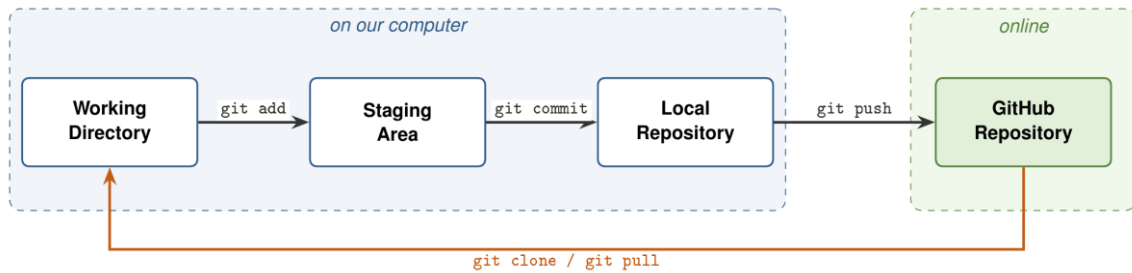


Figure 7: Basic Git workflow.

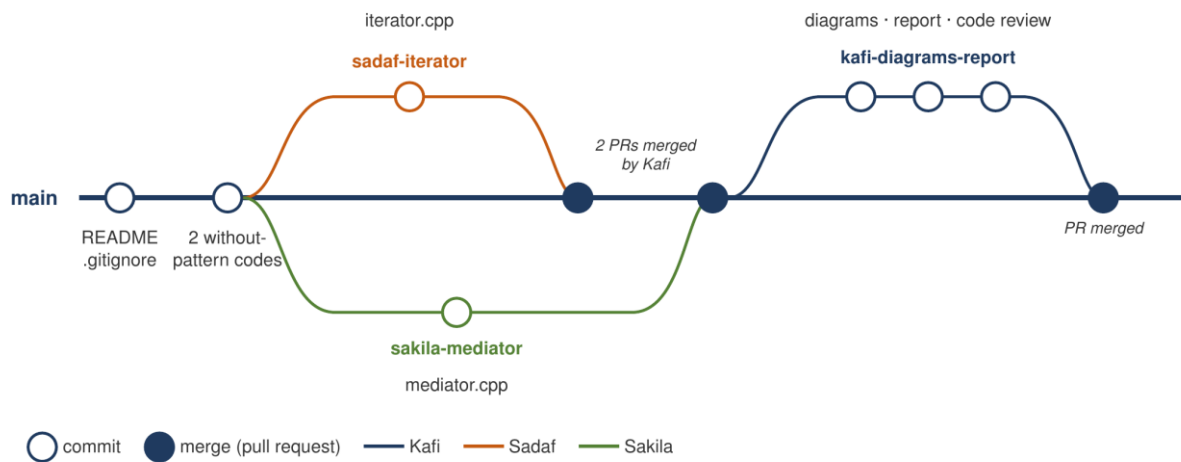


Figure 8: Branches of Group 07. Each member worked on a separate branch; Kafi merged them into main.

1. **Clone** – get a copy of the repository on our computer (`git clone`).
2. **Branch** – make our own branch so `main` stays safe (`git checkout -b sadaf-iterator`).
3. **Edit and test** – add the `.cpp` file and test it with `g++`.
4. **Stage and commit** – choose the files (`git add`) and save a snapshot with a message (`git commit -m "..."`).
5. **Push** – upload the branch to GitHub (`git push -u origin <branch>`).
6. **Pull request and review** – ask to join the branch into `main`; Kafi checks the code and approves it.
7. **Merge** – Kafi merges the pull request into `main`.
8. **Pull** – everyone downloads the latest `main` (`git pull origin main`).

4.4 Command-Line Usage

Example Git session (Sadaf's part)

```

Terminal
$ git clone https://github.com/Kafi2611/CSE3206-Lab3-Iterator-Mediator.git
$ cd CSE3206-Lab3-Iterator-Mediator
$ git checkout -b sadaf-iterator          # own branch
$ git add Iterator/iterator.cpp
$ git commit -m "Add Iterator pattern code (cafeteria menu)"
$ git push -u origin sadaf-iterator      # then open a pull request
$ git checkout main
$ git pull origin main                   # after Kafi merges
$ git log --oneline --graph              # see the history

```

4.5 Repository Screenshot

The screenshot shows the GitHub repository page for 'CSE3206-Lab3-Iterator-Mediator'. The repository is public and has 4 branches and 0 tags. The commit history shows a merge pull request #3 from Kafi2611/kafi-diagrams-report, with 8 commits. The commit history table is as follows:

File	Commit Message	Time Ago
Diagrams	Add diagrams and code review	now
Iterator	Add iterator pattern code	1 hour ago
Mediator	Add mediator pattern code	2 hours ago
.gitignore	Add without-pattern codes and fix .gitignore	6 hours ago
Code_Review.md	Add diagrams and code review	now
README.md	Add README and .gitignore	10 hours ago

The README file is titled 'CSE 3206 – Lab 3: Iterator and Mediator Design Patterns'. It describes the course and the lab's focus on Design Pattern Analysis, Implementation, and Code Review. The README includes a table with the following data:

Name	Student ID	GitHub	Work in this repo	Branch
Abdullah Al Kafi	2203139	@Kafi2611	Repository setup, README, both "without pattern" codes, UML diagrams, report, code review	main, kafi-diagrams-report
Sadaf Rahman	2203140	@sadaf532	Iterator Pattern code (iterator.cpp)	sadaf-iterator
Sakila Akter	2203141	@SakilaAkter	Mediator Pattern code (mediator.cpp)	sakila-mediator

The repository also shows the 'About' section, which includes the README, Activity, Stars, Watching, and Forks. The 'Releases' section shows no releases published. The 'Packages' section shows no packages published. The 'Contributors' section lists Kafi2611, sadaf532, and SakilaAkter. The 'Languages' section shows C++ at 100%.

Figure 9: GitHub repository of Group 07 showing the source files and commit history.

5 Comparison: Iterator vs Mediator

Both patterns are behavioral, but they solve different problems. Table 9 shows the main differences.

Point	Iterator	Mediator
Category	Behavioral	Behavioral
Main question	How do I visit every item without knowing how they are stored?	How can many objects work together without knowing each other?
Key idea	Move the “walking” logic into an iterator object.	Move the “talking” logic into a mediator object.
Removes coupling between	The client and the collection’s inside storage.	The colleague objects (many-to-many).
Main parts	Iterator, Concrete Iterator, Aggregate, Concrete Aggregate, Client	Mediator, Concrete Mediator, Colleague, Client
Shape	Client → Iterator → Collection	Many colleagues around one hub (a star)
Our example	Cafeteria menu (array + vector)	Group chat room (Kafi, Sadaf, Sakila)
Built into C++?	Yes – STL iterators and range-based for	No – we write it ourselves

Table 9: Iterator and Mediator side by side.

Mediator vs Observer (a common question)

In **Observer**, one subject sends updates to many listeners (one-to-many, one direction). In **Mediator**, many objects talk to each other *through* a middle object that can also apply rules (many-to-many through one hub). A chat room can use Observer inside it, but its main job is to be the mediator.

6 Conclusion

In this lab we studied, implemented and reviewed two behavioral design patterns. The **Iterator Pattern** let one `printMenu()` function print menus stored in an array and in a vector, without knowing how the items were stored. The **Mediator Pattern** let three users chat through one `ChatRoom` without a single pointer between them. In both cases a small object in the middle – the iterator or the mediator – removed direct dependencies, so the code became easier to change, test and reuse. Our code review also showed that small details, such as checking `hasNext()` before `next()` and managing memory safely, are important in real projects.

References

1. A. Shvets, *Dive Into Design Patterns*, 2018.
2. E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
3. E. Freeman and E. Robson, *Head First Design Patterns*, 2nd ed., O'Reilly Media, 2020.
4. Refactoring.Guru, “Iterator”, <https://refactoring.guru/design-patterns/iterator>
5. Refactoring.Guru, “Mediator”, <https://refactoring.guru/design-patterns/mediator>
6. cppreference.com, “Iterator library”, <https://en.cppreference.com/w/cpp/iterator>